

校准文件说明

1. 校准文件如何获取

D:\Nano\Cuga-Calibration\CalibrationResult\Result_日期时间.dat

- 按照日期进行分类排序存储的

2. 怎么验证校准对象是否可以使用

任何校准对象都是继承自 `CalibrationBase`，所以需要使用 `CalibrationBase.IsOk` 判断改校准对象是否能够使用

```
[Serializable]
public class CalibrationBase
{
    /// <summary>
    /// 是否校准Ok
    /// </summary>
    public bool IsCalibrated { get; set; }

    /// <summary>
    /// 是否验证Ok
    /// </summary>
    public bool IsVerified { get; set; }

    /// <summary>
    /// 是否Ok
    /// </summary>
    [JsonIgnore]
    public bool IsOk => IsCalibrated && IsVerified;
}
```

3. 有哪些大类

```
/// <summary>
/// 校准对象序列化
/// </summary>
[Serializable]
public sealed class CalibrationObj
{
    /// <summary>
    /// 缓震平台校准对象
    /// </summary>
    [Description(WcfConstantHelper.AdsNodeCalibrationName)]
    public CalibrationAdsObj CalibrationAdsObj { get; set; } = new CalibrationAdsObj();

    /// <summary>
    /// 显微镜校准对象
    /// </summary>
}
```

```

    /// </summary>
    [Description(WcfConstantHelper.MicroscopeNodeCalibrationName)]
    public CalibrationMicroscopeObj CalibrationMicroscopeObj { get; set; } = new
CalibrationMicroscopeObj();

    /// <summary>
    /// Chuck校准对象
    /// </summary>
    [Description(WcfConstantHelper.ChuckNodeCalibrationName)]
    public CalibrationChuckObj CalibrationChuckObj { get; set; } = new
CalibrationChuckObj();

    /// <summary>
    /// 激光校准对象
    /// </summary>
    [Description(WcfConstantHelper.LaserNodeCalibrationName)]
    public CalibrationLaserObj CalibrationLaserObj { get; set; } = new
CalibrationLaserObj();
}

```

1. 缓震平台: CalibrationAdsObj

```

    /// <summary>
    /// 缓震平台校准对象
    /// </summary>
    [Serializable]
    public sealed class CalibrationAdsObj
    {
        /// <summary>
        /// 压力前馈校准对象
        /// </summary>
        [Description(WcfConstantHelper.AdsPressureCalibrationName)]
        public CalibrationAdsPressureGains CalibrationAdsPressureGains { get; set; } = new
CalibrationAdsPressureGains();

        /// <summary>
        /// X方向速度前馈校准对象列表
        /// </summary>
        [Description(WcfConstantHelper.AdsXGainCalibrationName)]
        public CalibrationAdsXGainsItem[] CalibrationAdsXGainsItemList { get; set; } =
Array.Empty<CalibrationAdsXGainsItem>();

        /// <summary>
        /// y方向速度前馈校准对象列表
        /// </summary>
        [Description(WcfConstantHelper.AdsYGainCalibrationName)]
        public CalibrationAdsYGainsItem CalibrationAdsYGains { get; set; } = new
CalibrationAdsYGainsItem();
    }

```

1.1. 压力前馈校准: CalibrationAdsPressureGains

```
/// <summary>
/// 压力前馈校准对象
/// </summary>
[Serializable]
public sealed class CalibrationAdsPressureGains : CalibrationBase
{
    /// <summary>
    /// 比例阀输出值1(绝对), **需要下发ADS硬件**
    /// </summary>
    public double PressureValue1 { get; set; }

    /// <summary>
    /// 比例阀输出值2(绝对), **需要下发ADS硬件**
    /// </summary>
    public double PressureValue2 { get; set; }

    /// <summary>
    /// 比例阀输出值3(绝对), **需要下发ADS硬件**
    /// </summary>
    public double PressureValue3 { get; set; }
}
```

1.2. X方向速度前馈校准: CalibrationAdsXGainsItem

根据不同 `列表.SingleOrDefault(t => t.CgMagTypeEnum == 暗场Mag && t.Speed == 速度)` 判断 `is not null` 后使用:

- 注意下发的时候的正反向

个数: $3 * 3 = 9$

```
/// <summary>
/// X方向速度前馈校准对象
/// </summary>
[Serializable]
public sealed class CalibrationAdsXGainsItem : CalibrationBase
{
    /// <summary>
    /// 暗场Mag类型
    /// </summary>
    public CgMagTypeEnum CgMagTypeEnum { get; set; }

    /// <summary>
    /// 速度前馈校准速度
    /// </summary>
    public ADSSpeedEnum Speed { get; set; }

    /// <summary>
    /// 当前暗场Mag和速度下的X正向速度前馈系数x1(绝对), **需要下发ADS硬件**
    /// </summary>
}
```

```

public double PositiveX1 { get; set; }

/// <summary>
/// 当前暗场Mag和速度下的X正向前馈系数X2(绝对), **需要下发ADS硬件**
/// </summary>
public double PositiveX2 { get; set; }

/// <summary>
/// 当前暗场Mag和速度下的X反向前馈系数X3(绝对), **需要下发ADS硬件**
/// </summary>
public double NegativeX3 { get; set; }

/// <summary>
/// 当前暗场Mag和速度下的X反向馈系数X4(绝对), **需要下发ADS硬件**
/// </summary>
public double NegativeX4 { get; set; }
}

```

1.3. Y方向速度前馈校准: CalibrationAdsYGainsItem

- 注意下发的时候的正反向
- 目前方案只需要做100mm/s速度就行, 所以列表中只会有一条记录

```

/// <summary>
/// Y方向速度前馈校准对象
/// </summary>
[Serializable]
public sealed class CalibrationAdsYGainsItem : CalibrationBase
{
    /// <summary>
    /// Y方向平移台移动速度
    /// </summary>
    public double SpeedYValue { get; set; }

    /// <summary>
    /// 当前移动速度的Y正向前馈系数Y1(绝对), **需要下发ADS硬件**
    /// </summary>
    public double PositiveY1 { get; set; }

    /// <summary>
    /// 当前移动速度的Y正向前馈系数Y2(绝对), **需要下发ADS硬件**
    /// </summary>
    public double PositiveY2 { get; set; }

    /// <summary>
    /// 当前移动速度的Y正向前馈系数Y3(绝对), **需要下发ADS硬件**
    /// </summary>
    public double PositiveY3 { get; set; }

    /// <summary>
    /// 当前移动速度的Y反向前馈系数Y4(绝对), **需要下发ADS硬件**
    /// </summary>
}

```

```

public double NegativeY4 { get; set; }

/// <summary>
/// 当前移动速度的Y反向前馈系数Y5(绝对), **需要下发ADS硬件**
/// </summary>
public double NegativeY5 { get; set; }

/// <summary>
/// 当前移动速度的Y反向前馈系数Y6(绝对), **需要下发ADS硬件**
/// </summary>
public double NegativeY6 { get; set; }
}

```

2. 显微镜: CalibrationMicroscopeObj

```

/// <summary>
/// 显微镜校准对象
/// </summary>
[Serializable]
public sealed class CalibrationMicroscopeObj
{
    /// <summary>
    /// Focus校准对象列表
    /// </summary>
    [Description(WcfConstantHelper.MicroscopeFocusCalibrationName)]
    public CalibrationMicroscopeFocusItem[] CalibrationMicroscopeFocusItemList { get; set; }
    = Array.Empty<CalibrationMicroscopeFocusItem>();

    /// <summary>
    /// Cal Chip校准对象
    /// </summary>
    [Description(WcfConstantHelper.MicroscopeCalChipCalibrationName)]
    public CalibrationMicroscopeCalChip CalibrationMicroscopeCalChip { get; set; } = new
    CalibrationMicroscopeCalChip();

    /// <summary>
    /// PixelSize校准对象列表
    /// </summary>
    [Description(WcfConstantHelper.MicroscopePixelSizeCalibrationName)]
    public CalibrationMicroscopePixelSizeItem[] CalibrationMicroscopePixelSizeItemList {
    get; set; } = Array.Empty<CalibrationMicroscopePixelSizeItem>();

    /// <summary>
    /// Centricity校准对象列表
    /// </summary>
    [Description(WcfConstantHelper.MicroscopeCentricityCalibrationName)]
    public CalibrationMicroscopeCentricityItem[] CalibrationMicroscopeCentricityItemList {
    get; set; } = Array.Empty<CalibrationMicroscopeCentricityItem>();
}

```

2.1. 自动聚焦校准: CalibrationMicroscopeFocusItem

根据不同 `列表.SingleOrDefault(t => t.CgMicroscopeLens == 镜头)` 判断 `is not null` 后使用

个数: 5

```
/// <summary>
/// Focus校准对象
/// </summary>
[Serializable]
public sealed class CalibrationMicroscopeFocusItem : CalibrationBase
{
    /// <summary>
    /// 显微镜镜头
    /// </summary>
    public CgMicroscopeLens CgMicroscopeLens { get; set; }

    /// <summary>
    /// 当前镜头的最佳清晰高度(绝对ECS), **需要下发AF硬件**
    /// </summary>
    public double EcsValue { get; set; }

    /// <summary>
    /// 当前镜头的显微镜相像差镜头电压值(像差镜头绝对位置), **需要下发Microscope硬件**
    /// </summary>
    public double MicroscopeVoltage { get; set; }
}
```

2.2. Cal Chip4个耳朵自动聚焦校准: CalibrationMicroscopeCalChip

```
/// <summary>
/// Cal Chip校准对象
/// </summary>
[Serializable]
public sealed class CalibrationMicroscopeCalChip : CalibrationBase
{
    /// <summary>
    /// 此显微镜镜头下做的校准
    /// </summary>
    public CgMicroscopeLens CgMicroscopeLens { get; set; }

    /// <summary>
    /// 基于<see cref="CgMicroscopeLens"/>倍镜下做的校准, DSW明场中心的机械位置(绝对位置), **需要下发AF硬件**
    /// </summary>
    public CgPoint DswBrightFieldMachinePosition { get; set; }

    /// <summary>
    /// 基于<see cref="CgMicroscopeLens"/>倍镜下做的校准, DSW暗场中心的机械位置(绝对位置), **需要下发AF硬件**
    /// </summary>
    public CgPoint DswDarkFieldMachinePosition { get; set; }
}
```

```

    /// </summary>
    public CgPoint DswDarkFieldMachinePosition { get; set; }

    /// <summary>
    /// 基于<see cref="CgMicroscopeLens"/>倍镜下做的校准，DSW最佳清晰高度(绝对ECS)，**需要下发AF硬件**
    /// </summary>
    public double DswEcsValue { get; set; }

    /// <summary>
    /// 基于<see cref="CgMicroscopeLens"/>倍镜下做的校准，Undefined明场中心的机械位置(绝对位置)，**需要下发AF硬件**
    /// </summary>
    public CgPoint UndefinedBrightFieldMachinePosition { get; set; }

    /// <summary>
    /// 基于<see cref="CgMicroscopeLens"/>倍镜下做的校准，Undefined暗场中心的机械位置(绝对位置)，**需要下发AF硬件**
    /// </summary>
    public CgPoint UndefinedDarkFieldMachinePosition { get; set; }

    /// <summary>
    /// 基于<see cref="CgMicroscopeLens"/>倍镜下做的校准，Undefined最佳清晰高度(绝对ECS)，**需要下发AF硬件**
    /// </summary>
    public double UndefinedEcsValue { get; set; }

    /// <summary>
    /// 基于<see cref="CgMicroscopeLens"/>倍镜下做的校准，Haze明场中心的机械位置(绝对位置)，**需要下发AF硬件**
    /// </summary>
    public CgPoint HazeBrightFieldMachinePosition { get; set; }

    /// <summary>
    /// 基于<see cref="CgMicroscopeLens"/>倍镜下做的校准，Haze暗场中心的机械位置(绝对位置)，**需要下发AF硬件**
    /// </summary>
    public CgPoint HazeDarkFieldMachinePosition { get; set; }

    /// <summary>
    /// 基于<see cref="CgMicroscopeLens"/>倍镜下做的校准，Haze最佳清晰高度(绝对ECS)，**需要下发AF硬件**
    /// </summary>
    public double HazeEcsValue { get; set; }

    /// <summary>
    /// 基于<see cref="CgMicroscopeLens"/>倍镜下做的校准，Shinywafer明场中心的机械位置(绝对位置)，**需要下发AF硬件**
    /// </summary>
    public CgPoint ShinywaferBrightFieldMachinePosition { get; set; }

    /// <summary>

```

```

    /// 基于<see cref="CgMicroscopeLens"/>倍镜下做的校准，shinywafer暗场中心的机械位置(绝对位置)，
    **需要下发AF硬件**
    /// </summary>
    public CgPoint shinywaferDarkFieldMachinePosition { get; set; }

    /// <summary>
    /// 基于<see cref="CgMicroscopeLens"/>倍镜下做的校准，shinywafer最佳清晰高度(绝对ECS)，**需要
    下发AF硬件**
    /// </summary>
    public double shinywaferEcsValue { get; set; }
}

```

2.3. 像素尺寸校准: CalibrationMicroscopePixelSizeItem

根据不同 列表.SingleOrDefault(t => t.CgMicroscopeLens == 镜头) 判断 is not null 后使用

个数: 5

```

    /// <summary>
    /// PixelSize校准对象
    /// </summary>
    [Serializable]
    public sealed class CalibrationMicroscopePixelSizeItem : CalibrationBase
    {
        /// <summary>
        /// 显微镜镜头
        /// </summary>
        public CgMicroscopeLens CgMicroscopeLens { get; set; }

        /// <summary>
        /// 当前镜头的x方向和y方向1像素转尺寸，单位um/pixel(SizePerPixel)，**Cuga内部使用**
        /// </summary>
        public CgSize PixelSize { get; set; }
    }

```

2.4. 中心校准: CalibrationMicroscopeCentricityItem

根据不同 列表.SingleOrDefault(t => t.CgMicroscopeLens == 镜头) 判断 is not null 后使用

个数: 5

```

    /// <summary>
    /// Centricity校准对象
    /// </summary>
    [Serializable]
    public sealed class CalibrationMicroscopeCentricityItem : CalibrationBase
    {
        /// <summary>
        /// 显微镜镜头
        /// </summary>

```



```

public CgMicroscopeLens CgMicroscopeLens { get; set; }

/// <summary>
/// 当前镜头的中心位置偏差[相对距离](都是和150x偏差显微镜(当前镜头 - 150x镜头)，所以可以通过这个可以
计算任意两个镜头差)，**Cuga内部使用**
/// </summary>
public CgPoint Offset { get; set; }
}

```

3. Chuck: CalibrationChuckObj

```

/// <summary>
/// Chuck校准对象
/// </summary>
[Serializable]
public sealed class CalibrationChuckObj
{
    /// <summary>
    /// Gantry正交校准对象
    /// </summary>
    [Description(WcfConstantHelper.ChuckGantryCalibrationName)]
    public CalibrationChuckGantry CalibrationChuckGantry { get; set; } = new
    CalibrationChuckGantry();

    /// <summary>
    /// Chuck 全局比例误差校准对象
    /// </summary>
    [Description(WcfConstantHelper.ChuckGlobalScaleErrorCalibrationName)]
    public CalibrationChuckGlobalScaleError CalibrationChuckGlobalScaleError { get; set; } =
    new CalibrationChuckGlobalScaleError();

    /// <summary>
    /// Chuck Center校准对象
    /// </summary>
    [Description(WcfConstantHelper.ChuckCenterCalibrationName)]
    public CalibrationCenterObj CalibrationCenterObj { get; set; } = new
    CalibrationCenterObj();

    /// <summary>
    /// Prealigner校准对象
    /// </summary>
    [Description(WcfConstantHelper.ChuckPrealignerCalibrationName)]
    public CalibrationPrealignerObj CalibrationPrealignerObj { get; set; } = new
    CalibrationPrealignerObj();

    /// <summary>
    /// Stage Map 校准对象
    /// </summary>
    [Description(WcfConstantHelper.ChuckDarkFieldStageMapCalibrationName)]
    public CalibrationChuckDarkFieldStageMap CalibrationChuckDarkFieldStageMap { get; set; }
    = new CalibrationChuckDarkFieldStageMap();
}

```

```
}
```

3.1. 正交校准: CalibrationChuckGantry

```
/// <summary>
/// Gantry正交校准对象
/// </summary>
public sealed class CalibrationChuckGantry : CalibrationBase
{
    /// <summary>
    /// 此显微镜镜头下做的校准
    /// </summary>
    public CgMicroscopeLens CgMicroscopeLens { get; set; }

    /// <summary>
    /// 基于<see cref="CgMicroscopeLens"/>倍镜下做的校准，整个Y轴的误差(绝对偏差)，**需要下发ACS硬件**
    /// </summary>
    public double offset { get; set; }
}
```

3.2. 比例校准: CalibrationChuckGlobalScaleError

```
/// <summary>
/// 全局比例误差校准对象
/// </summary>
[Serializable]
public sealed class CalibrationChuckGlobalScaleError : CalibrationBase
{
    /// <summary>
    /// 此显微镜镜头下做的校准
    /// </summary>
    public CgMicroscopeLens CgMicroscopeLens { get; set; }

    /// <summary>
    /// 基于<see cref="CgMicroscopeLens"/>倍镜下，X轴比例误差系数，**Cuga内部使用**
    /// </summary>
    public double ScaleX { get; set; }

    /// <summary>
    /// 基于<see cref="CgMicroscopeLens"/>倍镜下，Y轴比例误差系数，**Cuga内部使用**
    /// </summary>
    public double ScaleY { get; set; }
}
```

3.3. Chuck Center校准: CalibrationCenterObj

```
/// <summary>
/// Chuck Center校准对象
/// </summary>
```

```
[Serializable]
public sealed class CalibrationCenterObj : CalibrationBase
{
    /// <summary>
    /// 此显微镜镜头下做的校准
    /// </summary>
    public CgMicroscopeLens CgMicroscopeLens { get; set; }

    /// <summary>
    /// 基于<see cref="CgMicroscopeLens"/>倍镜下做的校准，校准后明场中心Stage的物理坐标，**Cuga内部使用以及下发AF硬件**
    /// </summary>
    public CgPoint NewBFCenterStagePosition { get; set; }
}
```

3.4. Prealigner校准: CalibrationPrealignerObj

```
/// <summary>
/// Prealigner校准对象
/// </summary>
[Serializable]
public sealed class CalibrationPrealignerObj : CalibrationBase
{
    /// <summary>
    /// 此显微镜镜头下做的校准
    /// </summary>
    public CgMicroscopeLens CgMicroscopeLens { get; set; }

    /// <summary>
    /// 基于<see cref="CgMicroscopeLens"/>倍镜下做的校准，校准后EFEM上下料位置的Stage物理坐标，**Cuga内部使用**
    /// </summary>
    public CgPoint NewEfemLoadWaferStagePosition { get; set; }

    /// <summary>
    /// 基于<see cref="CgMicroscopeLens"/>倍镜下做的校准，校准后EFEM上下料时Chuck的起始旋转角度，**Cuga内部使用**
    /// </summary>
    public double NewEfemLoadWaferChuckAngle { get; set; }
}
```

3.5. Stage Map校准: CalibrationChuckDarkFieldStageMap

使用 `ExpandStageMap.ToCgErrorMap()` 下发

```
/// <summary>
/// 暗场 Stage Map 校准对象
/// </summary>
[Serializable]
public sealed class CalibrationChuckDarkFieldStageMap : CalibrationBase
```

```

{
    /// <summary>
    /// 此显微镜镜头下做的校准
    /// </summary>
    public CgMicroscopeLens CgMicroscopeLens { get; set; }

    /// <summary>
    /// 此暗场Mag类型下做的校准
    /// </summary>
    public CgMagTypeEnum OpticsMagTypeEnum { get; set; }

    /// <summary>
    /// 此速度下做的校准
    /// </summary>
    public ADSSpeedEnum Speed { get; set; }

    /// <summary>
    /// 基于<see cref="CgMicroscopeLens"/>倍镜, <see cref="OpticsMagTypeEnum"/>Mag, <see
    cref="Speed"/>速度下做的校准. StageMap矩阵(笛卡尔坐标系), **需要下发ACS硬件**
    /// </summary>
    public StageMap ExpandStageMap { get; set; }
}

/// <summary>
/// StageMap矩阵(笛卡尔坐标系)
/// </summary>
[Serializable]
public sealed class StageMap
{
    /// <summary>
    /// 理想矩阵的起点
    /// </summary>
    public CgPoint IdealBasePoint { get; set; }

    /// <summary>
    /// 行数量
    /// </summary>
    public int RowNumber { get; set; }

    /// <summary>
    /// 列数量
    /// </summary>
    public int ColumnNumber { get; set; }

    /// <summary>
    /// 列宽
    /// </summary>
    public double ColumnCellWidth { get; set; }

    /// <summary>
    /// 行高
    /// </summary>
    public double RowCellHeight { get; set; }
}

```

```

    /// <summary>
    /// 误差矩阵
    /// </summary>
    public CgPoint[][] ErrorMatrix { get; set; }

    /// <summary>
    /// 转换为Cuga下发ACS硬件的ErrorMap
    /// </summary>
    /// <returns>Cuga下发ACS硬件的ErrorMap</returns>
    public CgErrorMap ToCgErrorMap()
    {
        return new CgErrorMap
        {
            Zone = 0,
            BaseX = IdealBasePoint.X,
            BaseY = IdealBasePoint.Y,
            XStep = ColumnCellWidth,
            YStep = RowCellHeight,
            Rows = Enumerable.Range(0, RowNumber).Select(row => new CgErrorMapRow
            {
                Id = row,
                Cols = Enumerable.Range(0, ColumnNumber).Select(column => new CgErrorMapCol
                {
                    Id = column,
                    Location = ErrorMatrix[row][column]
                }).ToList()
            }).ToList()
        };
    }
}

```

4. 激光: CalibrationLaserObj

```

    /// <summary>
    /// 激光校准对象
    /// </summary>
    [Serializable]
    public sealed class CalibrationLaserObj
    {
        /// <summary>
        /// 暗场自动聚焦，AB两路灯亮度校准对象
        /// </summary>
        [Description(WcfConstantHelper.LaserAutoFocusCalibrationName)]
        public CalibrationLaserAutoFocus CalibrationLaserAutoFocus { get; set; } = new
        CalibrationLaserAutoFocus();

        /// <summary>
        /// 台面功率计校准对象
        /// </summary>
        [Description(WcfConstantHelper.LaserOpticalPowerCalibrationName)]
    }

```

```

    public CalibrationLaserOpticalPower[] CalibrationLaserOpticalPowerList { get; set; } =
Array.Empty<CalibrationLaserOpticalPower>();

    /// <summary>
    /// AOD延迟时间校准对象列表
    /// </summary>
    [Description(WcfConstantHelper.LaserAodDelayCalibrationName)]
    public CalibrationLaserAodDelayItem[] CalibrationLaserAodDelayItemList { get; set; } =
Array.Empty<CalibrationLaserAodDelayItem>();

    /// <summary>
    /// 均匀性校准对象
    /// </summary>
    [Description(WcfConstantHelper.LaserIlluminationProfileCalibrationName)]
    public CalibrationLaserIlluminationProfileItem[]
CalibrationLaserIlluminationProfileItemList { get; set; } =
Array.Empty<CalibrationLaserIlluminationProfileItem>();

    /// <summary>
    /// XTC
    /// </summary>
    [Description(WcfConstantHelper.LaserXtcCalibrationName)]
    public CalibrationLaserXTCCalibrationItem[] CalibrationLaserXtcCalibrationItemList {
get; set; } = Array.Empty<CalibrationLaserXTCCalibrationItem>();

    /// <summary>
    /// 暗场相机的Y像素尺寸校准对象列表
    /// </summary>
    [Description(WcfConstantHelper.LaserPixelSizeCalibrationName)]
    public CalibrationLaserPixelSizeItem[] CalibrationLaserPixelSizeItemList { get; set; } =
Array.Empty<CalibrationLaserPixelSizeItem>();

    /// <summary>
    /// XPixelSizer校准对象
    /// </summary>
    [Description(WcfConstantHelper.LaserXPixelSizeCalibrationName)]
    public CalibrationLaserXPixelSizeItem[] CalibrationLaserXPixelSizeList { get; set; } =
Array.Empty<CalibrationLaserXPixelSizeItem>();

    /// <summary>
    /// 暗场相机的像素尺寸校准对象列表
    /// </summary>
    [Description(WcfConstantHelper.LaserLineCentricityCalibrationName)]
    public CalibrationLaserLineCentricityItem[] CalibrationLaserLineCentricityItemList {
get; set; } = Array.Empty<CalibrationLaserLineCentricityItem>();
}

```

4.1. 自动聚焦校准: CalibrationLaserAutoFocus

```

/// <summary>
/// 暗场自动聚焦，AB两路灯亮度校准对象
/// </summary>

```

```
[Serializable]
public sealed class CalibrationLaserAutoFocus : CalibrationBase
{
    /// <summary>
    /// A路灯的电流值(绝对电流值), **需要下发AF硬件**
    /// </summary>
    public double CurrentA { get; set; }

    /// <summary>
    /// B路灯的电流值(绝对电流值), **需要下发AF硬件**
    /// </summary>
    public double CurrentB { get; set; }
}
```

4.2. 台面功率计校准: CalibrationLaserOpticalPower

根据不同 列表.SingleOrDefault(t => t.CgMagTypeEnum == 暗场Mag) 判断 is not null 后使用

个数: 3

```
/// <summary>
/// 台面功率计校准对象
/// </summary>
[Serializable]
public sealed class CalibrationLaserOpticalPower : CalibrationBase
{
    /// <summary>
    /// Mag类型
    /// </summary>
    public CgMagTypeEnum CgMagTypeEnum { get; set; }

    /// <summary>
    /// 当前暗场Mag的测量的最大功率, **Cuga内部使用**
    /// </summary>
    public double MeasureMaxPower { get; set; }

    /// <summary>
    /// 当前暗场Mag的测量的最大功率机械坐标, **Cuga内部使用**
    /// </summary>
    public CgPoint MeasureMaxPowerPosition { get; set; }
}
```

4.3. Aod延迟校准: CalibrationLaserAodDelayItem

根据不同 列表.SingleOrDefault(t => t.CgMagTypeEnum == 暗场Mag) 判断 is not null 后使用

个数: 3

```
/// <summary>
/// AOD延迟时间校准对象
/// </summary>
public sealed class CalibrationLaserAodDelayItem : CalibrationBase
```

```

{
    /// <summary>
    /// 暗场Mag类型
    /// </summary>
    public CgMagTypeEnum CgMagTypeEnum { get; set; }

    /// <summary>
    /// 当前暗场Mag的Prescan AOD 延迟时间(绝对延迟时间), **需要下发Laser硬件**
    /// </summary>
    public double PrescanAodDelayTime { get; set; }

    /// <summary>
    /// 当前暗场Mag的Chirp AOD 延迟时间(绝对延迟时间), **需要下发Laser硬件**
    /// </summary>
    public double ChirpAodDelayTime { get; set; }
}

```

4.4. 待定 AOD 散光校准

4.5. AOD Prescan均匀性校准:

CalibrationLaserIlluminationProfileItem

根据不同 `列表.SingleOrDefault(t => t.Coefficient == 幅值 && t.CgMagTypeEnum == 暗场Mag)` 判断
`is not null` 后使用

个数: $13 * 3 = 39$

```

/// <summary>
/// 均匀性校准对象
/// </summary>
[Serializable]
public sealed class CalibrationLaserIlluminationProfileItem : CalibrationBase
{
    /// <summary>
    /// 波形功率系数(1表示100%, 0表示0%)
    /// </summary>
    public double Coefficient { get; set; }

    /// <summary>
    /// Mag类型
    /// </summary>
    public CgMagTypeEnum OpticsMagTypeEnum { get; set; }

    /// <summary>
    /// 当前暗场Mag和功率系数下的结果prescan文件路径, **需要下发Laser硬件**
    /// </summary>
    public string ResultPrescanFilePath { get; set; }

    /// <summary>
    /// 当前暗场Mag和功率系数下的P偏振功率, **Cuga内部使用**
    /// </summary>
    public double PolarizationPPower { get; set; }
}

```



```

    /// <summary>
    /// 当前暗场Mag和功率系数下的S偏振功率，**Cuga内部使用**
    /// </summary>
    public double PolarizationSPower { get; set; }

    /// <summary>
    /// 当前暗场Mag和功率系数下的C偏振功率，**Cuga内部使用**
    /// </summary>
    public double PolarizationCPower { get; set; }
}

```

4.6. CIB的采样窗口完全同步校准: CalibrationLaserXTCCalibrationItem

根据不同 `列表.SingleOrDefault(t => t.CgMagTypeEnum == 暗场Mag && t.PmtId == PmtId)` 判断 `is not null` 后使用

个数: $3 * 15 = 45$

```

    /// <summary>
    /// LaserXTCCalibration
    /// </summary>
    [Serializable]
    public sealed class CalibrationLaserXTCCalibrationItem : CalibrationBase
    {
        /// <summary>
        /// Mag类型
        /// </summary>
        public CgMagTypeEnum CgMagTypeEnum { get; set; }

        /// <summary>
        /// 暗场相机ID
        /// </summary>
        public int PmtId { get; set; }

        /// <summary>
        /// 当前暗场Mag和PmtId下的通道1延迟时间，**需要下发Laser硬件**
        /// </summary>
        public int CH1Delay { get; set; }

        /// <summary>
        /// 当前暗场Mag和PmtId下的通道2延迟时间，**需要下发Laser硬件**
        /// </summary>
        public int CH2Delay { get; set; }

        /// <summary>
        /// 当前暗场Mag和PmtId下的通道3延迟时间，**需要下发Laser硬件**
        /// </summary>
        public int CH3Delay { get; set; }
    }

```

```
}
```

4.7. 暗场相机Y像素尺寸校准: CalibrationLaserPixelSizeItem

根据不同 `列表.SingleOrDefault(t => t.CgMagTypeEnum == 暗场Mag && t.PmtId == PmtId)` 判断 `is not null` 后使用

个数: $3 * 15 = 45$

```
/// <summary>
/// 暗场相机的Y像素尺寸校准对象
/// </summary>
[Serializable]
public sealed class CalibrationLaserPixelSizeItem : CalibrationBase
{
    /// <summary>
    /// 暗场Mag类型
    /// </summary>
    public CgMagTypeEnum CgMagTypeEnum { get; set; }

    /// <summary>
    /// 暗场相机ID
    /// </summary>
    public int PmtId { get; set; }

    /// <summary>
    /// 当前暗场Mag和PmtId下的y方向1像素转尺寸, 单位um/pixel, **Cuga内部使用**
    /// </summary>
    public double YPixelSize { get; set; }
}
```

4.8. 暗场相机X像素尺寸校准: CalibrationLaserXPixelSizeItem

根据不同 `列表.SingleOrDefault(t => t.CgMagTypeEnum == 暗场Mag && t.Speed == 速度)` 判断 `is not null` 后使用

个数: $3 * 3 = 9$

```
/// <summary>
/// 暗场相机的X像素尺寸校准对象
/// </summary>
[Serializable]
public sealed class CalibrationLaserXPixelSizeItem : CalibrationBase
{
    /// <summary>
    /// Mag类型
    /// </summary>
    public CgMagTypeEnum CgMagTypeEnum { get; set; }
```

```

    /// <summary>
    /// 速度
    /// </summary>
    public ADSSpeedEnum Speed { get; set; }

    /// <summary>
    /// 当前暗场Mag和速度下的x方向1像素转尺寸，单位um/pixel，**Cuga内部使用**
    /// </summary>
    public double xPixelSize { get; set; }
}

```

4.9. 明暗场中心的offset校准:

CalibrationLaserLineCentricityItem

根据不同 列表.SingleOrDefault(t => t.CgMagTypeEnum == 暗场Mag && t.PmtId == PmtId && t.Speed == 速度) 判断 is not null 后使用

个数: $3 * 3 * 15 = 135$

```

    /// <summary>
    /// 中心与明场中心的offset校准
    /// </summary>
    [Serializable]
    public sealed class CalibrationLaserLineCentricityItem : CalibrationBase
    {
        /// <summary>
        /// 此显微镜镜头下做的校准
        /// </summary>
        public CgMicroscopeLens CgMicroscopeLens { get; set; }

        /// <summary>
        /// Mag类型
        /// </summary>
        public CgMagTypeEnum CgMagTypeEnum { get; set; }

        /// <summary>
        /// 速度
        /// </summary>
        public ADSSpeedEnum Speed { get; set; }

        /// <summary>
        /// 暗场相机ID
        /// </summary>
        public int PmtId { get; set; }

        /// <summary>
        /// 当前暗场Mag和速度PmtId下的基于<see cref="CgMicroscopeLens"/>倍镜下，正向暗场中心坐标，
        /// **Cuga内部使用，8号光斑需要下发到AF硬件**
        /// </summary>
        public CgPoint DarkMachineCenterPosition { get; set; }

        /// <summary>

```

```
/// 当前暗场Mag和速度PmtId下的基于<see cref="CgMicroscopeLens"/>倍镜下，反向暗场中心坐标，  
**Cuga内部使用**  
/// </summary>  
public CgPoint ReverseDarkMachineCenterPosition { get; set; }  
}
```

4.10. 待定 Pmt Gain
